System Design

Full-Stack Web Developers

System Design ≠ Software Architecture ≠ Design Patterns

System Design focuses on how **entire systems** operate and scale, **Software Architecture** focuses on how applications are **structured internally**, and **Design Patterns** focus on reusable solutions to recurring **code-level problems**.

System Design is the **art** of **moving data** efficiently through a system while making the right **trade-offs** for **current** and **future** scale.

Trade-offs > “Best” Solutions

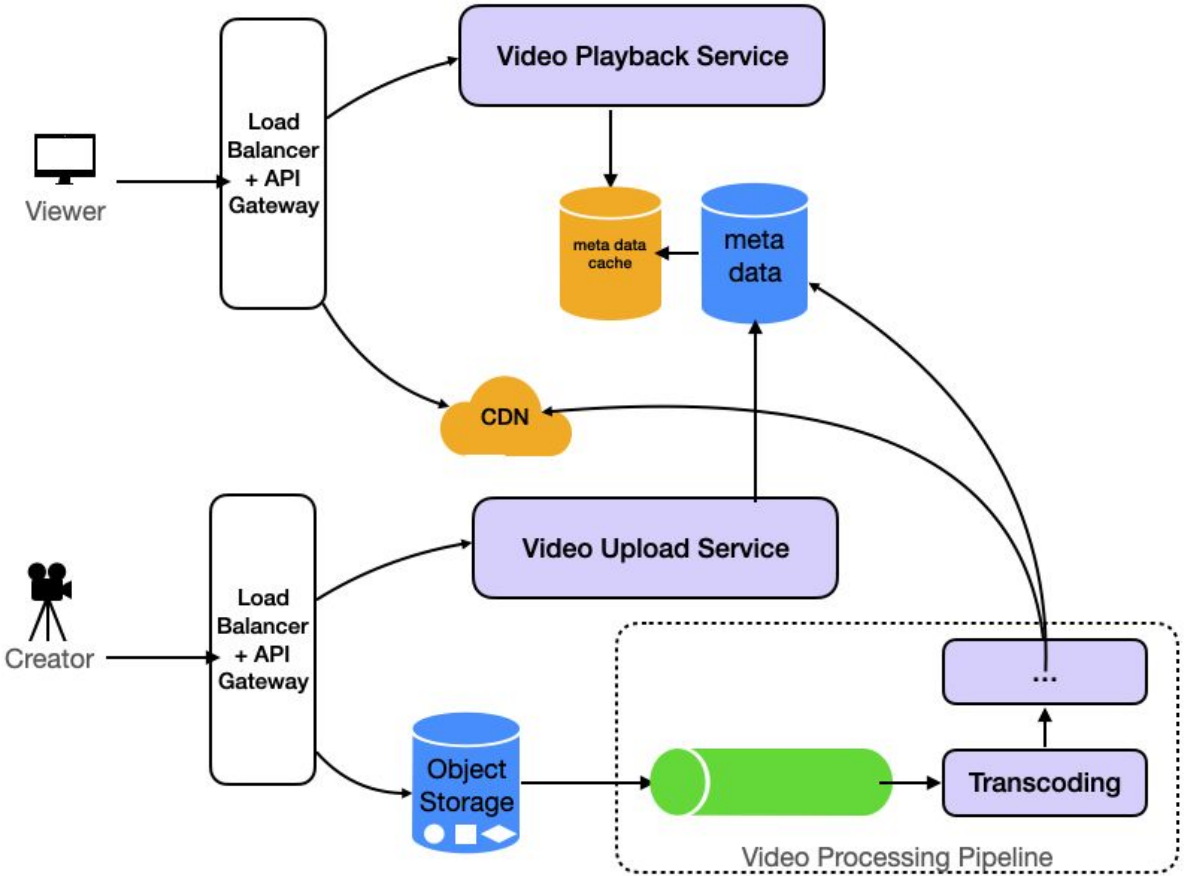
There is no perfect architecture — only trade-offs.

Data Flow is Everything

Most systems are just data moving through components.

Scale Changes Everything
(But Start Simple)

Design for today’s scale, but understand tomorrow’s problems.


Video Playback Service
Load Balancer + API Gateway
Viewer
meta data cache
meta data
CDN
Video Upload Service
Load Balancer + API Gateway
Creator
Object Storage
...
Transcoding
Video Processing Pipeline

System Design Diagrams follow conventions that prioritize **clear abstraction**, **readable data flow**, and **understandable infrastructure relationships** rather than strict universal notation rules.

For junior candidates:

Clear reasoning beats fancy architecture.

Aim for clean simple design with:

- good explanations
- sensible trade-offs
- awareness of limitations

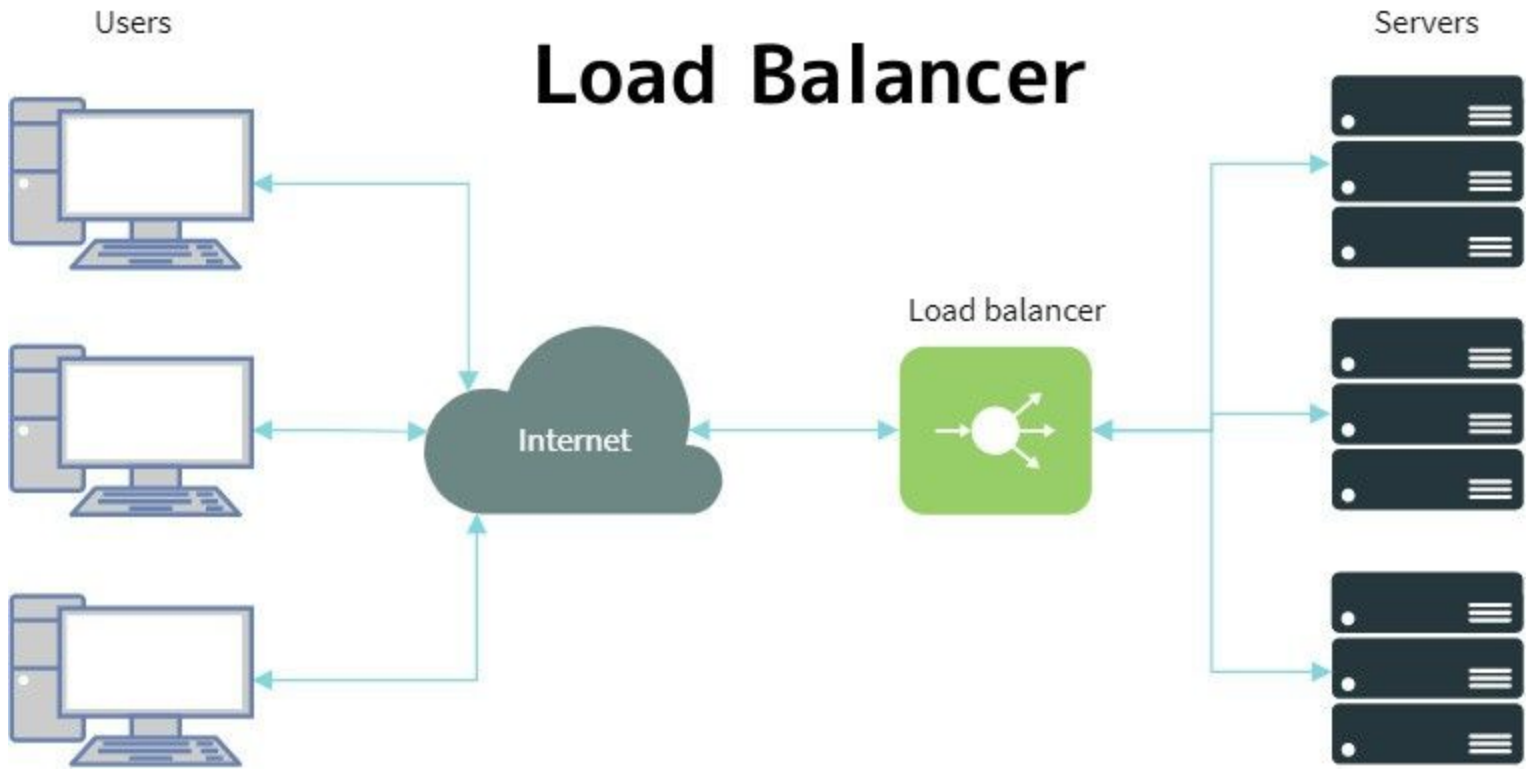
Engineering is **constraint** management.

Key Concepts:

1. Vertical vs Horizontal scaling
2. Caching (Redis, CDN)
3. Load balancing
4. Database bottlenecks

Load Balancer

A **load balancer** is like a traffic controller that spreads visitors across multiple workers so no single worker becomes overwhelmed.

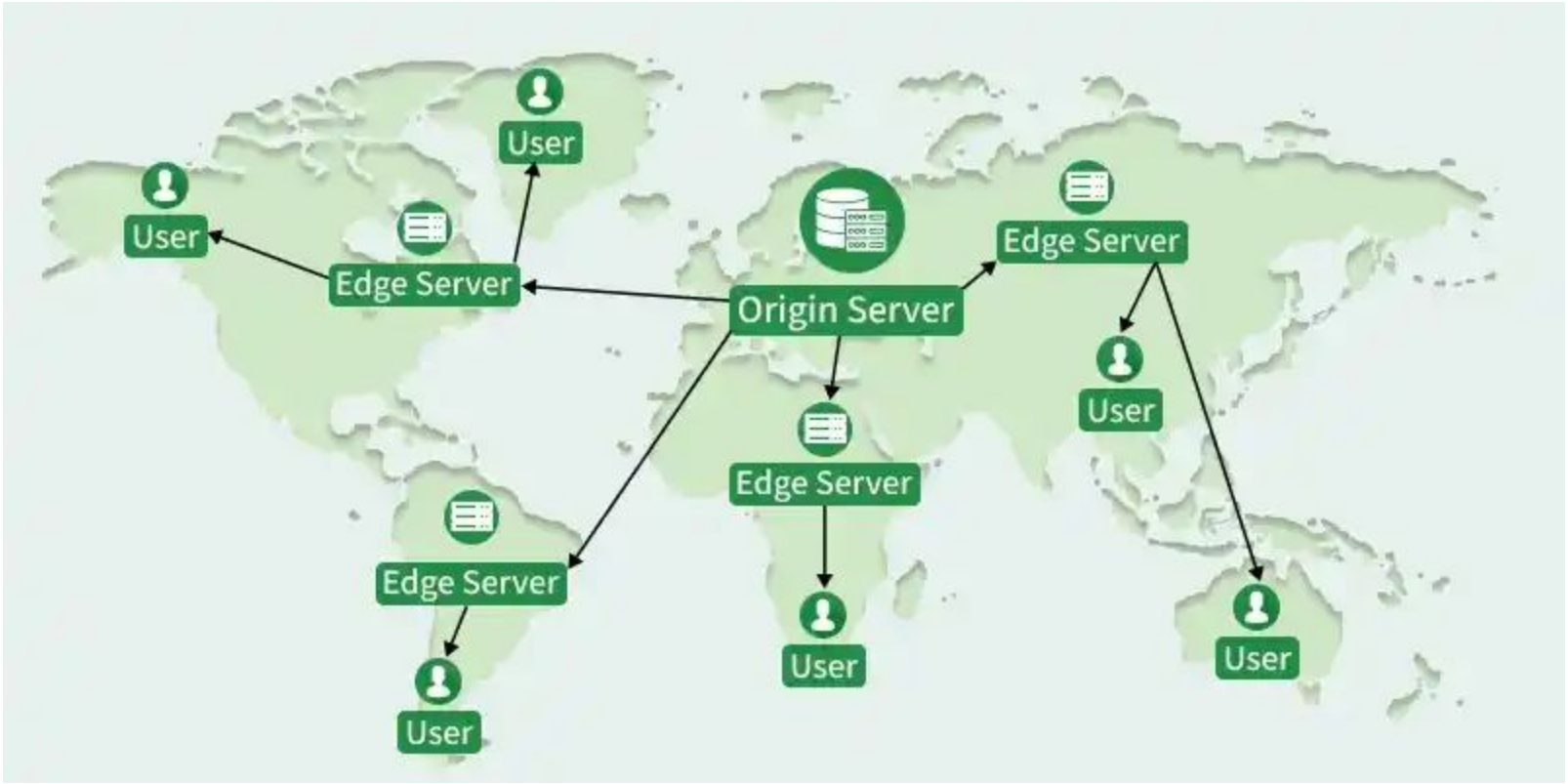

Users
Load Balancer
Servers
Load balancer
Internet

A **load balancer** is a network component that distributes incoming requests across multiple backend servers to improve **scalability**, **availability**, and **reliability**.

CDN
(Content Delivery Network)

A **CDN** is like having copies of a website stored in many locations around the world so people can access it faster from nearby.


User
User
Origin Server
User


User
User
Edge Server
Origin Server
Edge Server
User
Edge Server
Edge Server
User
User
User

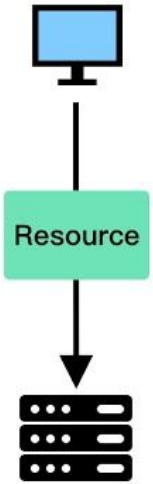
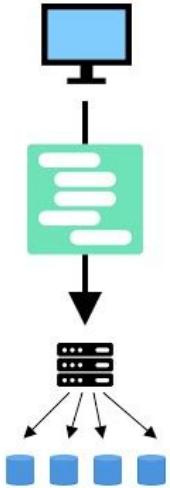
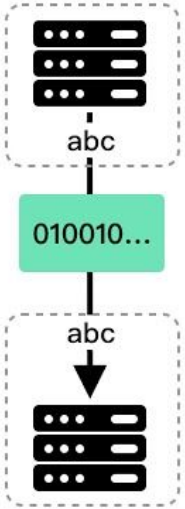
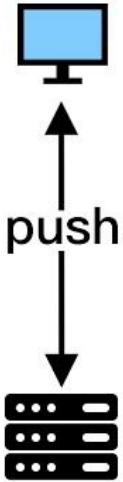
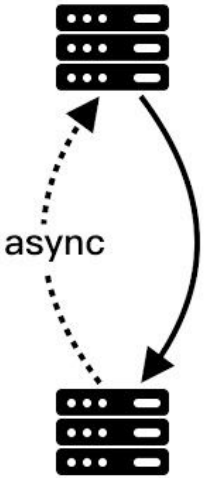
A **CDN** is a geographically distributed network of **edge servers** that caches and delivers content closer to users to **reduce latency** and **improve performance**.

API Design

Full-Stack Web Developers

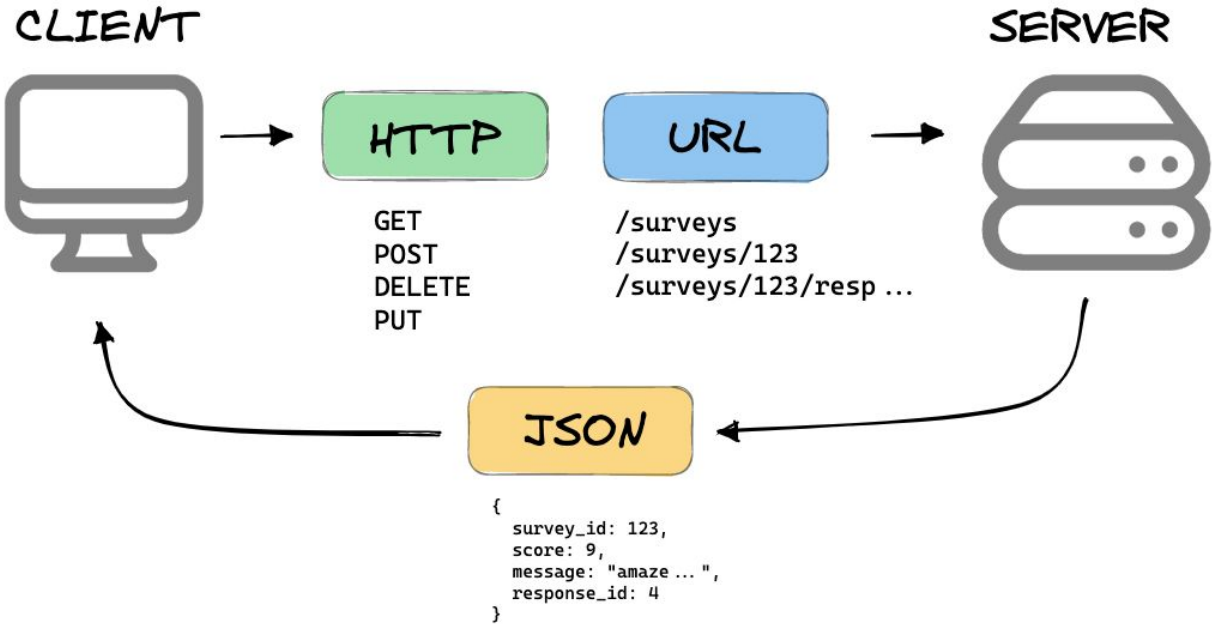
API Design is mainly about making communication between systems predictable, maintainable, secure, and developer-friendly.

Top 6 Most Popular API Architecture Patterns

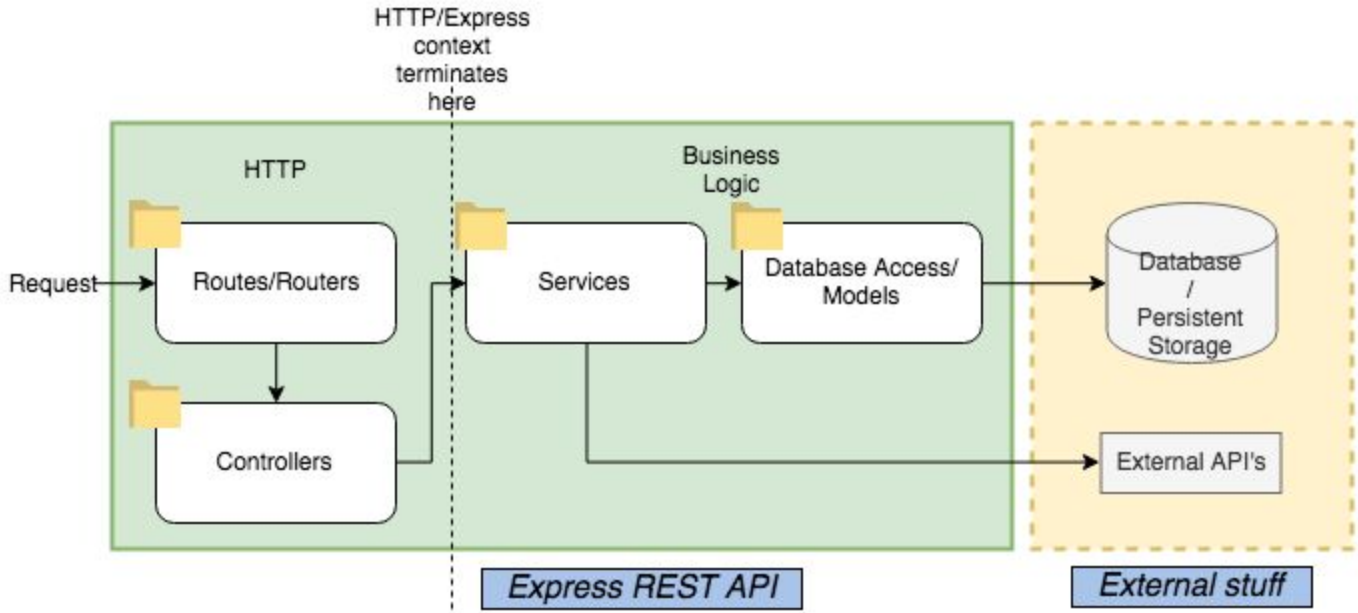
Style	SOAP	RESTful	GraphQL	gRPC	WebSocket	Webhook
Illustration						

RESTful API

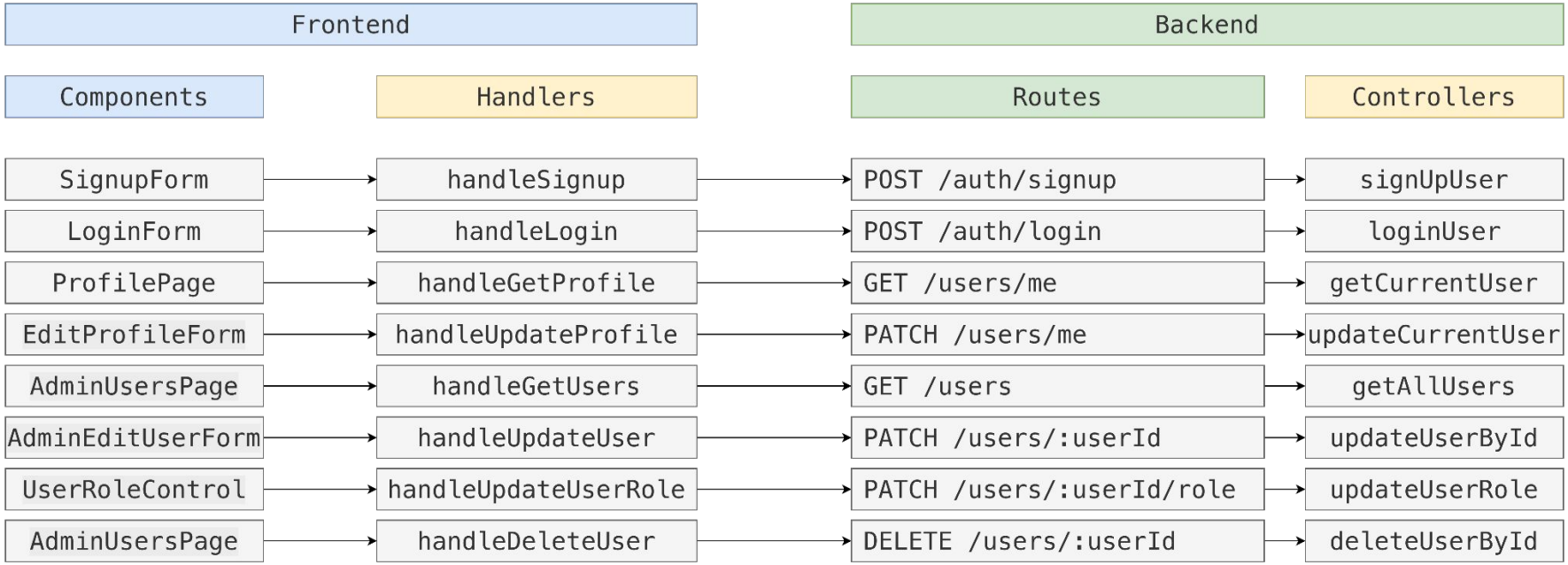
WHAT IS A REST API?



mannhowie.com


HTTP/Express
context
terminates
here
HTTP
Business
Logic
Request
Routes/Routers
Controllers
Services
Database Access/
Models
Database
/
Persistent
Storage
External API's
Express REST API
External stuff

Endpoint	Method	Purpose
/auth/signup	POST	Register a new user
/auth/login	POST	Authenticate and issue JWT
/users/me	GET	Get logged-in user profile from JWT
/users/me	PATCH	Update logged-in user’s own details
/users	GET	List all users, admin only
/users/:userId	PATCH	Admin update user details
/users/:userId/role	PATCH	Admin change user role
/users/:userId	DELETE	Admin delete user


Frontend
Backend
Components
Handlers
Routes
Controllers
SignupForm
handleSignup
POST /auth/signup
signUpUser
LoginForm
handleLogin
POST /auth/login
loginUser
ProfilePage
handleGetProfile
GET /users/me
getCurrentUser
EditProfileForm
handleUpdateProfile
PATCH /users/me
updateCurrentUser
AdminUsersPage
handleGetUsers
GET /users
getAllUsers
AdminEditUserForm
handleUpdateUser
PATCH /users/:userId
updateUserById
UserRoleControl
handleUpdateUserRole
PATCH /users/:userId/role
updateUserRole
AdminUsersPage
handleDeleteUser
DELETE /users/:userId
deleteUserById

Frontend names can **describe actions** like createNote, but **backend API routes** should **describe resources** like /notes, using HTTP methods to describe the action.

Watch/Read and (ค่อยๆ สบายๆ) Understand These Videos/Articles:

1. System Design → Google-style Interview
2. RESTful API
3. Design API like Senior Engineers

🏁 Today’s Challenge:

In your **project group**, work together in a virtual board (like Figjam) to:

1. Draw the **System Design diagram** for your project.
2. Create the **API Specification** for your project.
3. Create the **Frontend–Backend Interaction Diagram** for your project.

Optional:

- Go behind Vercel/Render abstraction and visualize the complete system functionality.
- Understand why **Vercel** for **frontend** and why **Render** for **backend**?

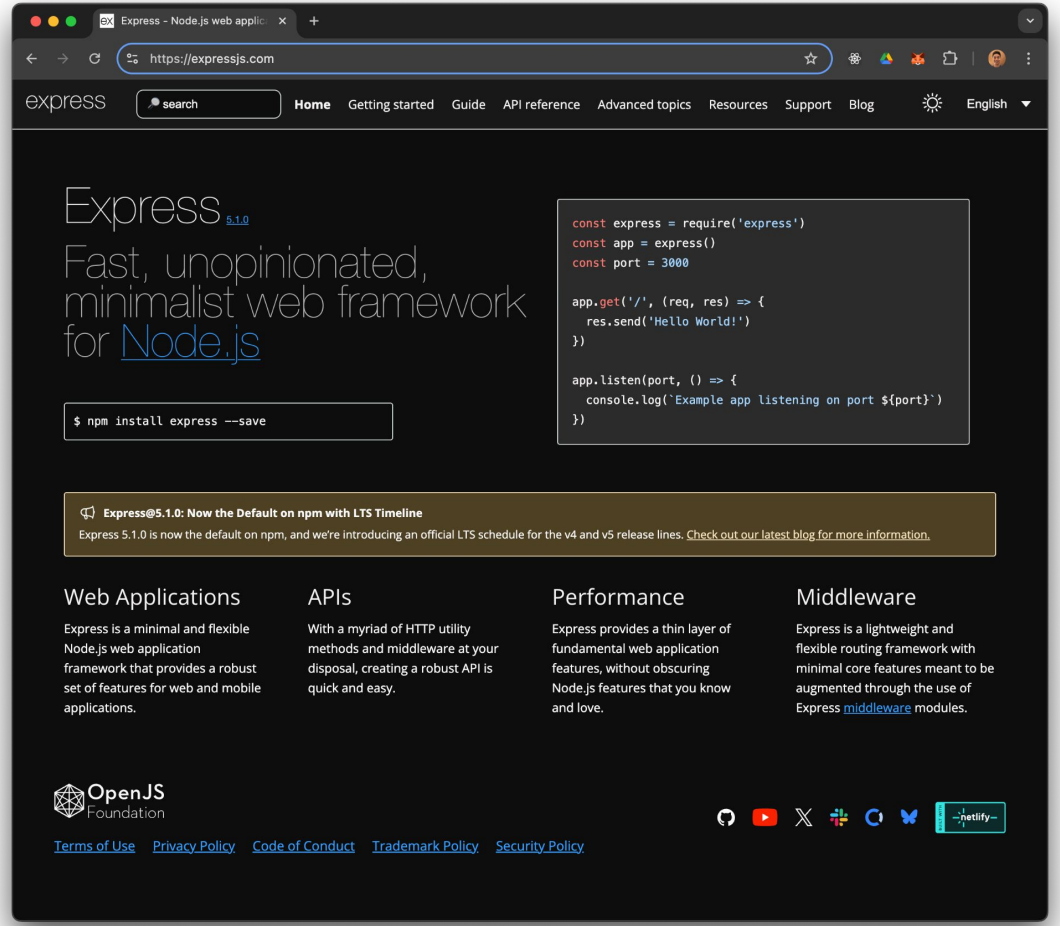
Submission

1. Submit Virtual board URL here

2. Share understanding

API Server

Full-Stack Web Developers


Express - Node.js web applic
https://expressjs.com
express
search
Home
Getting started
Guide
API reference
Advanced topics
Resources
Support
Blog
English
Express 5.1.0
Fast, unopinionated, minimalist web framework for Node.js
$ npm install express --save
const express = require('express')
const app = express()
const port = 3000
app.get('/', (req, res) => {
res.send('Hello World!')
})
app.listen(port, () => {
console.log(`Example app listening on port ${port}`)
})
Express@5.1.0: Now the Default on npm with LTS Timeline
Express 5.1.0 is now the default on npm, and we're introducing an official LTS schedule for the v4 and v5 release lines. Check out our latest blog for more information.
Web Applications
Express is a minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.
APIs
With a myriad of HTTP utility methods and middleware at your disposal, creating a robust API is quick and easy.
Performance
Express provides a thin layer of fundamental web application features, without obscuring Node.js features that you know and love.
Middleware
Express is a lightweight and flexible routing framework with minimal core features meant to be augmented through the use of Express middleware modules.
OpenJS Foundation
Terms of Use
Privacy Policy
Code of Conduct
Trademark Policy
Security Policy
netlify

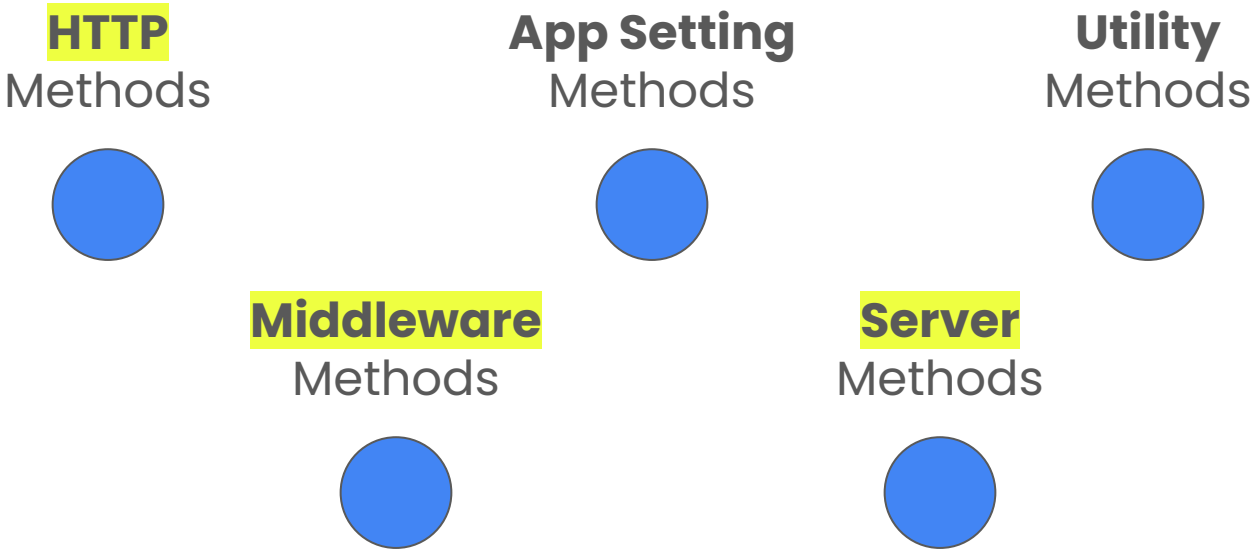
server.js

```
import express from “express”


const app = express()
```

In Express.js, the express() function is a **factory function** that creates an Express application **object**.

This object is a JavaScript function (a **request** handler) but and **has a bunch of methods and properties** attached to it **for building web servers**.

Here are the main methods of the app object …



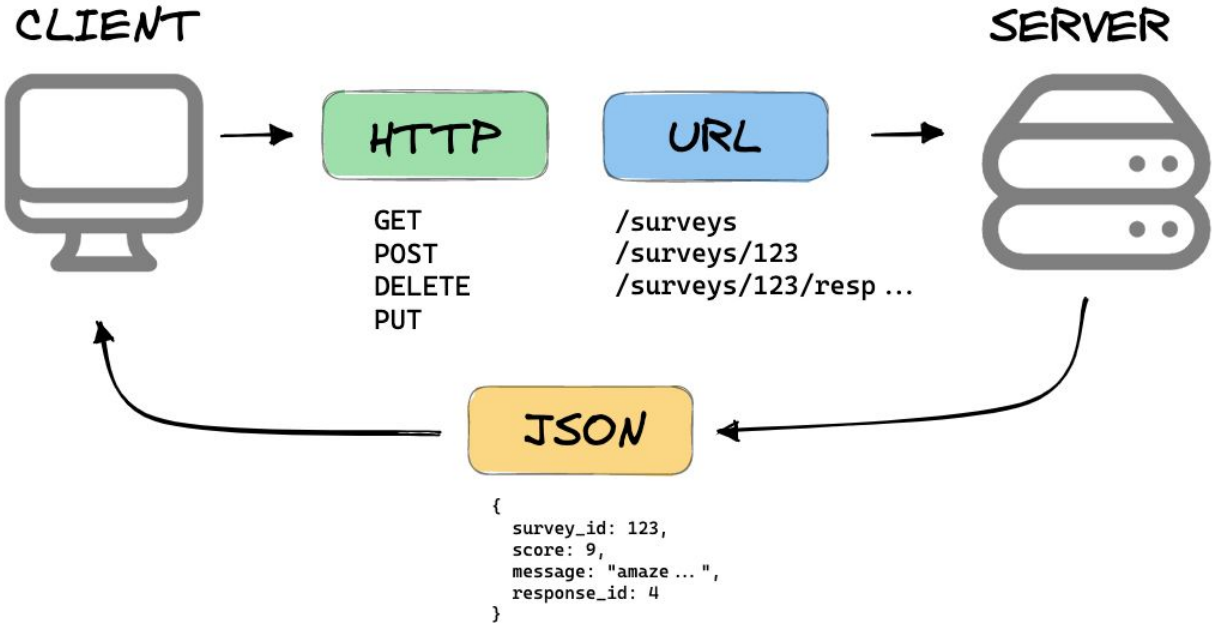
The most common **methods** you'll use are:

- **Routing**: `.get()`, `.post()`, `.put()`, `.delete()`
- **Middleware**: `.use()`
- **Server start**: `.listen()`
- **Config**: `.set()`

RESTful API

REST stands for **Representational State Transfer** — an **architectural style** for designing APIs where resources (like users, notes, products) are represented and manipulated via **standard HTTP** methods in a **stateless**, **consistent**, and **predictable** way.

WHAT IS A REST API?



mannhowie.com

In REST, **representation state** means the data about **a resource** at a given moment, expressed in a specific format (like JSON, XML, or HTML), which the server sends to the client.

Resource = the actual thing on the server (e.g., a “note” in a database).

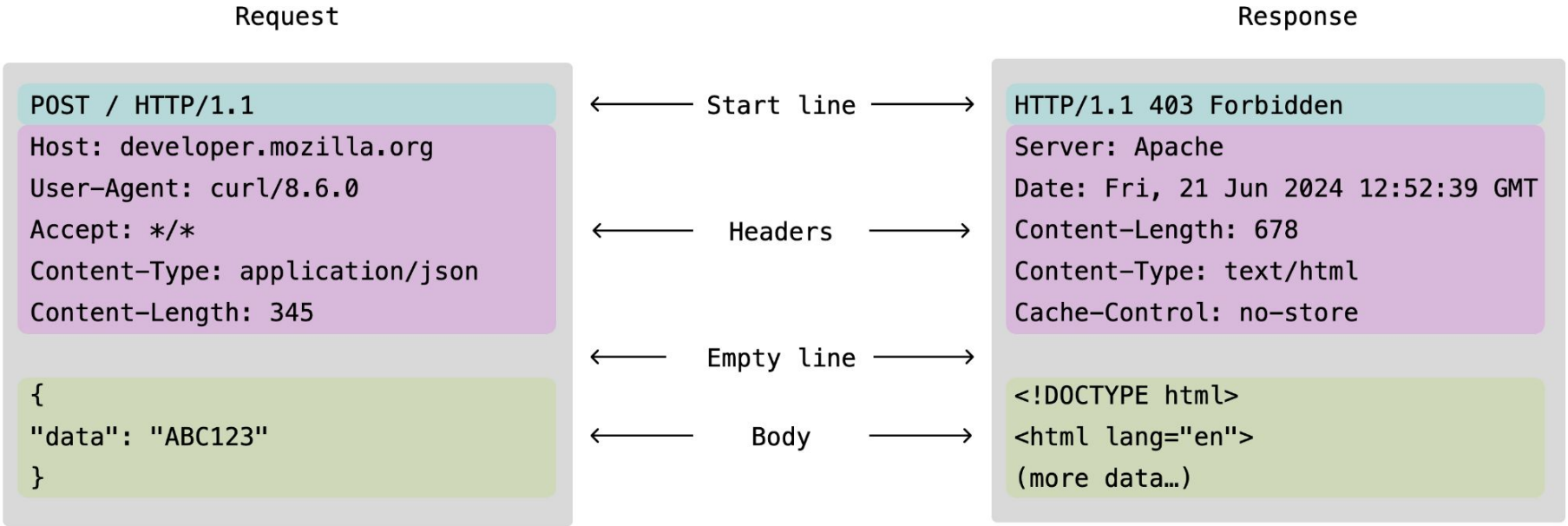
Representation = how that **resource’s current state** is presented to the client (e.g., { "id": 1, "title": "Buy milk" }).

The client manipulates the state of the resource by sending or receiving these representations **through HTTP requests and responses**.

👉 In short: REST **doesn’t send the resource itself**, but a representation of its state that the client can read or modify.

Structure of:

HTTP Request and Response


Request
Response
POST / HTTP/1.1
Host: developer.mozilla.org
User-Agent: curl/8.6.0
Accept: */*
Content-Type: application/json
Content-Length: 345
{
"data": "ABC123"
}
Start line
Headers
Empty line
Body
HTTP/1.1 403 Forbidden
Server: Apache
Date: Fri, 21 Jun 2024 12:52:39 GMT
Content-Length: 678
Content-Type: text/html
Cache-Control: no-store
<!DOCTYPE html>
<html lang="en">
(more data…)

📁

server.js

```
import express from “express”

const app = express()

app.get('/', (req, res) => {
    res.send(“Hello React, I am your server!”)
})

const PORT = 3000

app.listen(PORT, () => {
    console.log(`Server listening on port ${PORT} ✅ 🙌`)
})
```

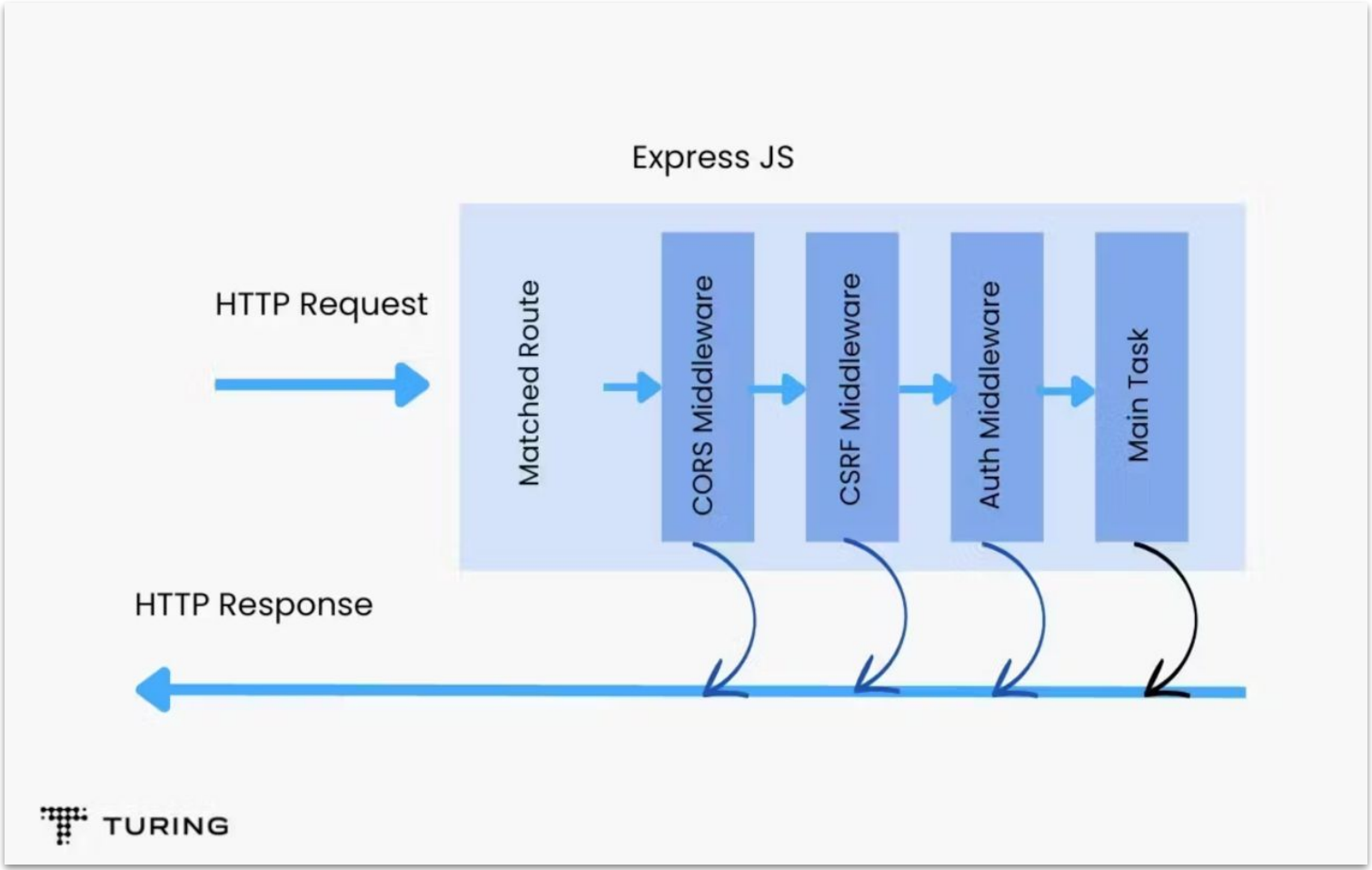
Properties of:

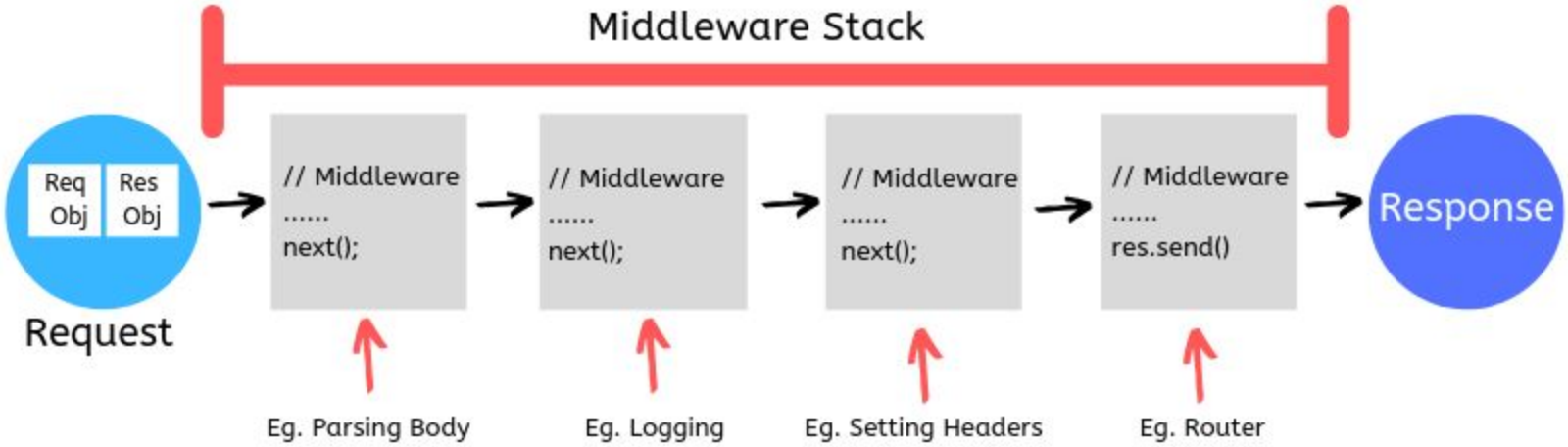
`req` and `res` objects

middlewares

Middlewares

Middleware in Express is like a **series of helpers** that **handle requests step by step**—checking, preparing, or modifying things—before the app gives a final **response**.


Express JS
HTTP Request
Matched Route
CORS Middleware
CSRF Middleware
Auth Middleware
Main Task
HTTP Response
TURING


Middleware Stack
Req Obj
Res Obj
Request
// Middleware
.....
next();
// Middleware
.....
next();
// Middleware
.....
next();
// Middleware
.....
res.send()
Response
Eg. Parsing Body
Eg. Logging
Eg. Setting Headers
Eg. Router

.json()

The .json() middleware in Express is **like a translator** that takes information written in JSON (a common data format) and **turns it into plain JavaScript** so the app can easily understand and use it.

API Routes

📁

server.js

```
// ...

let members = [];


app.post("/members", (req, res) => {
  const { name, lastname, position } = req.body;


  const newMember = {
    id: String(members.length + 1),
    name: name,
    lastname: lastname,
    position: position,
  };

  members.push(newMember);

  res.status(201).json(newMember);
});

// ...
```

Common HTTP status codes:

200: Successful request, often a GET

201: Successful request after a create, usually a POST

204: Successful request with no content returned, usually a PUT or PATCH

301: Permanently redirect to another endpoint

400: Bad request (client should modify the request)

401: Unauthorized, credentials not recognized

403: Forbidden, credentials accepted but don't have permission

404: Not found, the resource does not exist

410: Gone, the resource previously existed but does not now

429: Too many requests, used for rate limiting and should include retry headers

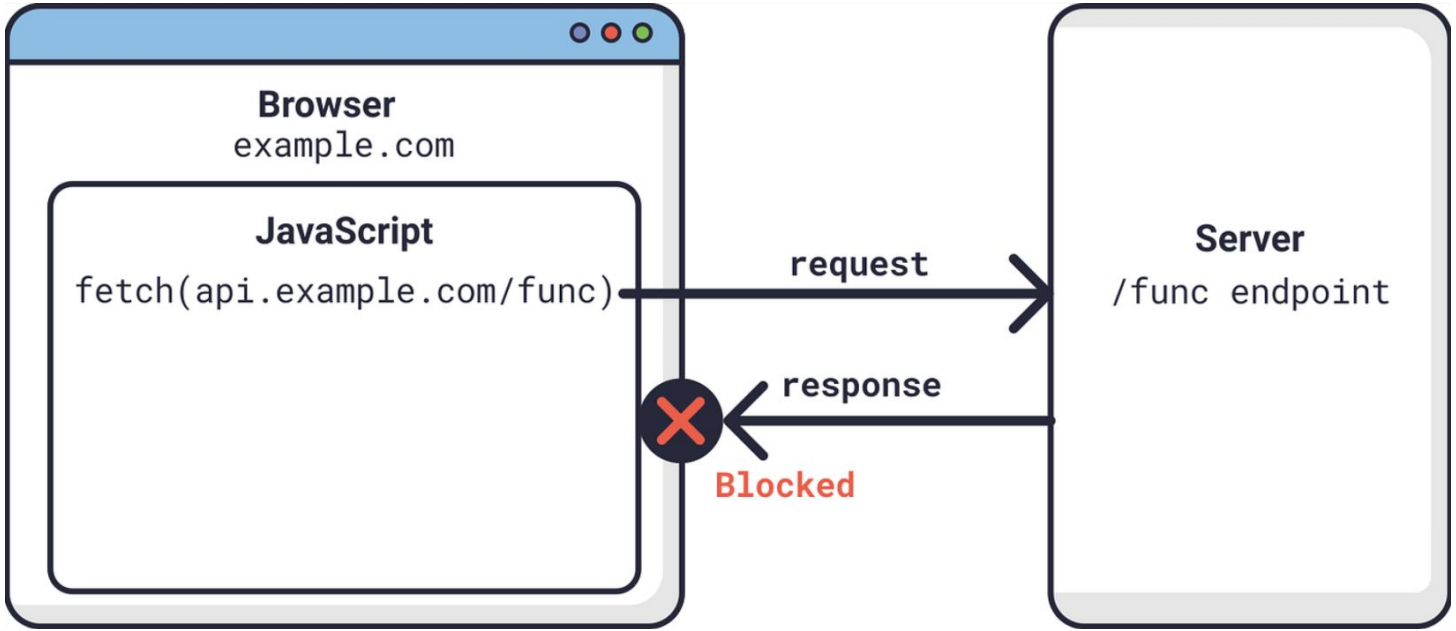
500: Server error, generic and worth looking at other 500-level errors instead

503: Service unavailable, another where retry headers are useful

CORS

The cors middleware in Express is like a **gatekeeper** that decides **which outside websites are allowed** to talk to your app.

The browser is the enforcer of cors not the server!



Cross-Origin Resource Sharing (CORS)

CORS protects the user not the server!

Cross-Origin Resource Sharing (CORS)

The browser's main incentive with CORS is protecting your personal data on other websites.

The "Bank Account" Problem

To understand why the browser blocks these requests, imagine this scenario:

You are logged into your bank: You have a tab open for mybank.com. Your browser is holding a "session cookie" that proves you are logged in.

You visit a malicious site: In another tab, you visit evil-site.com (maybe you clicked a bad link).

The Attack: evil-site.com has a script that tries to run a fetch("https://mybank.com/transfer-money").

Without the Same-Origin Policy (and CORS):

The browser would see that request, attach your bank's login cookies to it, and the bank would think you made the request. The malicious site could drain your account just because you had the tab open.

cors: allowing cookies from the client

📁express-02

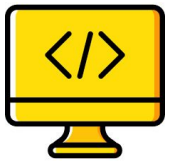
server.js

```
// ...

const corsOptions = {
  origin: [
    "http://localhost:5173",
    "http://localhost:5174",
    "http://localhost:5175",
    "https://rag-notes.vercel.app",
  ], // frontend domain
  credentials: true, // ✅ allow cookies to be sent
};

// ...
```

API Versioning

Thank you & Well done!

Happy Coding!



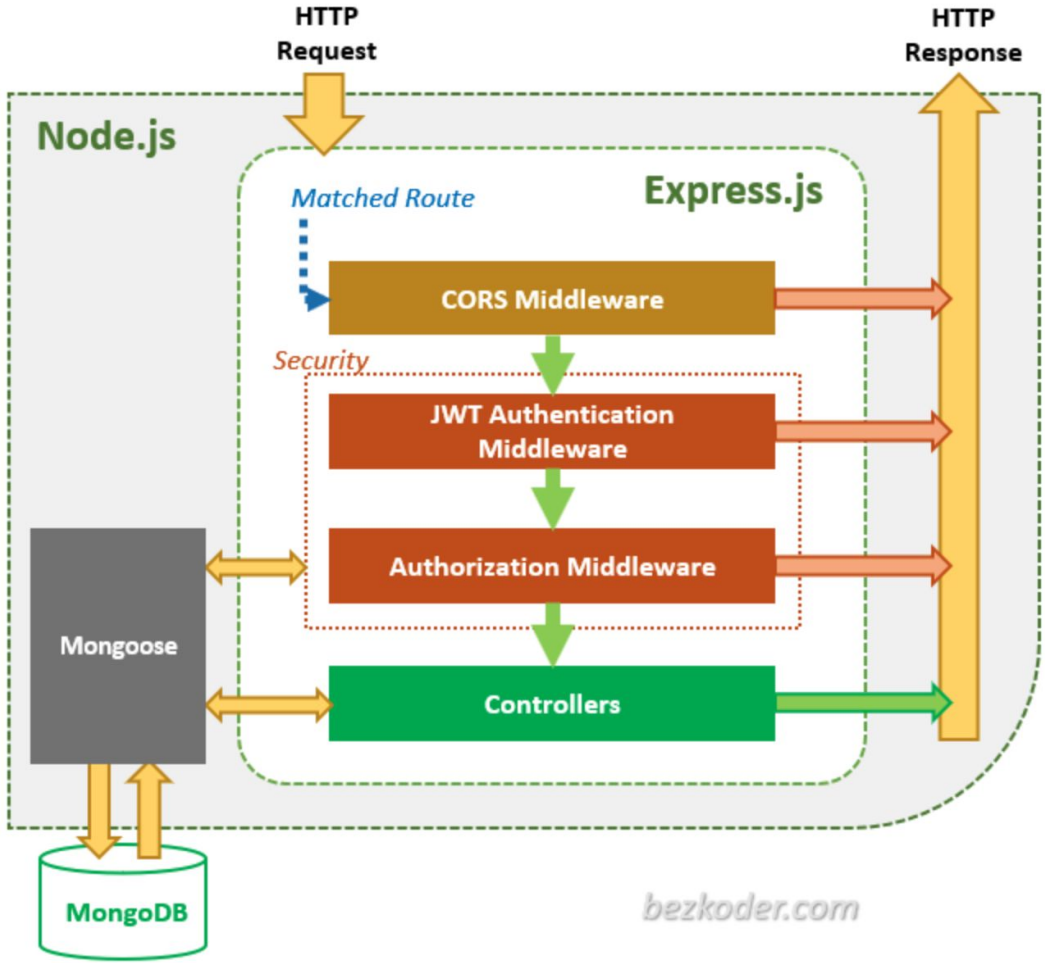
One piece of advice 🧙

“Stay consistent, stay curious, and *most importantly* , enjoy problem solving with or without code!”

Error Handling

Connect the Database

MongoDB and Mongoose


HTTP
Request
HTTP
Response
Node.js
Express.js
Matched Route
CORS Middleware
Security
JWT Authentication
Middleware
Authorization Middleware
Mongoose
Controllers
MongoDB
bezkoder.com

Major Categories of Browser APIs

The DOM (Document Object Model) API

This is the most fundamental Browser API. it allows you to treat an HTML document as a tree structure of objects.

What it does: You use it to add, remove, or change HTML elements and CSS styles dynamically.

Common methods: document.querySelector(), addEventListener(), createElement().

Communication APIs

These allow the browser to talk to external servers without refreshing the page.

Fetch API: The modern standard for making HTTP requests (replacing the older XMLHttpRequest).

WebSockets: Allows for two-way, real-time communication (like in a chat app).

Storage and State APIs

These allow you to save data locally on the user's machine so it persists even if they close the tab.

localStorage / sessionStorage: Simple key-value pairs for saving small amounts of data.

IndexedDB: A full-blown client-side database for complex data.

Cookie API: Used for session management and tracking.

Hardware and Device APIs

These bridge the gap between web software and physical hardware.

Geolocation API: Asks for permission to access the user's GPS coordinates.

Web Audio API: For high-level audio processing and synthesis.

Canvas & WebGL: For rendering 2D and 3D graphics using the computer’s GPU.

WebHID/WebBluetooth: For connecting to physical peripherals like game controllers or heart rate monitors.

User Experience APIs

History API: Lets you manipulate the browser’s back/forward history (crucial for Single Page Applications).

Notification API: Allows the browser to send system-level pop-ups.

Intersection Observer: Detects when an element is visible on the screen (great for lazy-loading images).